

MyHarnessPy — 架构设计文档

基于 Python 的轻量级 Agent 运行时框架，参照 Hermes Agent 八层架构范式实现。

目录

1. 整体分层概览
2. 各层详细说明
3. 感知·思考·行动·反馈 映射
4. 关键数据流
5. 核心设计原则

整体分层概览

入口层 Entry Layer	安全层
CLI / Web UI / REST Client / 外部触发器	Security Boundary
会话与路由层 Session & Routing Layer	AUTH
GW (REST/WS) → SessionRunner → SessionStore	用户授权
CommandGuards (cancel / state validation / persona)	
Agent Runtime Core	APPROVAL
AIAgent 主循环	危险命令审批
PROMPT 组装 调模型 执行工具 回填结果	SHELL SAFETY
COMPRESS 上下文压缩 持久化状态 返回输出	参数数组防注入
能力层 Capability Layer	OUTPUT LIMITS
工具系统 Skills Persistent Memory	溢出引用保护
执行层 Execution Backends	PERSONA GUARD
Terminal FileSystem External API Sub-agents	工具权限边界
状态层 State & Persistence	
SQLite (sessions / checkpoints) JSONL Messages MemoryStore	
模型层 Model / Provider Layer	
Provider Runtime Resolver → anthropic_messages / chat_completions / compatible	

各层详细说明

1. 入口层 Entry Layer

入口层只负责接收输入、展示输出、处理平台协议，不包含任何业务逻辑。

入口	实现	职责
CLI / TUI	cli.py	交互式命令行，支持 --persona 、 --provider 、 /skill-name 等指令

入口	实现	职责
Web UI	static/index.html	单页应用，通过 REST + WebSocket 与后端通信，展示消息流
REST Client	api/rest.py (入口侧)	任何 HTTP 客户端、脚本、第三方服务的接入点

入口层不感知 session 内部状态，只负责将外部请求转交给会话与路由层。

2. 会话与路由层 Session & Routing Layer

负责将各入口的请求**统一成内部会话格式**，路由给对应 Agent，并处理会话级控制逻辑。

Gateway（api/rest.py + api/ws.py）

平台协议转换层，将 HTTP / WebSocket 消息规范化后转发：

```
POST /sessions/{id}/messages → engine.send_message(text)
POST /sessions/{id}/cancel   → engine.cancel()
GET  /sessions/{id}/state    → engine.get_snapshot()
ws:// /ws/{session_id}      → 双向实时通道
```

SessionRunner（harness/engine/engine.py）

每个 session 对应一个 AgentEngine 实例，负责：

- 持有消息列表 _messages 和会话上下文
- 维护 asyncio.Lock 保护并发状态读写
- 管理 asyncio.Event 取消信号
- 管理 _intervention_queue（RUNNING 中收到的消息入队，轮末 drain）

SessionStore（harness/storage/）

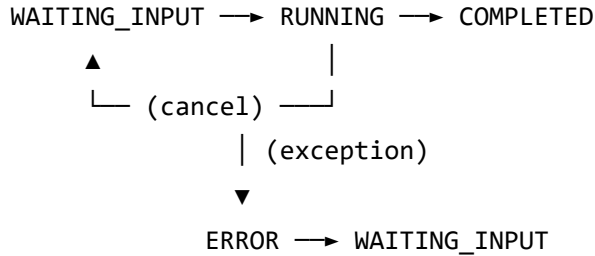
会话持久化与恢复：

- session.py — 抽象接口
- backends/sqlite.py — SQLite 持久化实现
- backends/memory.py — 内存实现（测试用）

支持 session 中断后从 SQLite 恢复完整消息历史。

CommandGuards (harness/engine/state_machine.py)

状态机强制校验合法转换：



- cancel → WAITING_INPUT (取消是预期行为，不是错误)
- 非法转换由 assert_legal_transition() 硬拒绝

Persona 路由

Persona 在会话创建时决定三件事：**系统提示词 + 工具权限 + LLM Provider**，相当于会话级的路由配置。

3. Agent Runtime Core

框架真正的核心。AIAgent 主循环驱动 ReAct 推理，每轮严格按顺序执行：

接收消息

- PROMPT 组装 (系统提示词 + Skill 摘要 + 历史上下文)
- 压缩检查 (COMPRESS: Micro / Auto 双层)
- 调模型 (provider.chat → assistant_msg)
- 执行工具 (execute_all → tool_result_msg)
- 回填结果 (ToolResultBlock 追加到 _messages)
- 必要时再次压缩
- 持久化状态 (storage.save)
- 返回输出 (WebSocket 推送 / REST 轮询)

PROMPT 组装 (harness/skills.py)

- load_persona(name) — 加载 Persona 系统提示词
- build_skill_system_addendum(skills) — 将所有 Skill 描述追加到提示词末尾

- 最终系统提示 = persona.system_prompt + skill_descriptions

主循环（ harness/engine/loop.py ） — 每轮 10 步

步骤	操作
1	Cancel check — 取消信号立即终止
2	COMPRESS — Micro / Auto 压缩检查
3	Tool Discovery — registry.discover() ， 每轮重载，不缓存
4	LLM Call — provider.chat(messages, tools)
5	Text-only check — 无工具调用则结束轮次
6	Loop Detection — 重复模式检测，注入换思路提示
7	Tool Execution — asyncio.gather() 并发执行
8	Protocol Validate — validate_message_sequence()
9	Atomic Append — assistant_msg + tool_result_msg 一次性追加
10	Drain Queue — 注入干预队列中的用户消息

COMPRESS （ harness/engine/compression.py ）

策略	触发条件	操作
Micro	消息数 > micro_keep_recent × 4	清空旧 ToolResultBlock 内容，无 LLM 开销
Auto	估算 token > window × 65%	小模型生成摘要替换旧消息， 压缩后强制重注入系统提示词 + 任务目标

其他 Core 组件

- LoopDetector （ loop_detector.py ） — SHA-1 指纹 + 5 轮滑动窗口，重复模式自动注入突破提示
- MsgValidator （ types/messages.py ） — validate_message_sequence() 强制 ToolCall ↔ ToolResult 配对
- EventEmitter （ observability/events.py ） — 结构化 JSON 日志，四态决策模型

4. 能力层 Capability Layer

能力层不决定 Agent 如何思考，但决定 Agent 能调用什么、记住什么、复用什么。

工具系统（harness/tools/）

ToolRegistry + ToolExecutor 提供统一的工具注册与并发执行框架。内置工具覆盖终端、文件系统、搜索等场景；工具输出超限时自动走 OverflowStore 引用机制，避免消息膨胀。

Skills（skills/<name>/SKILL.md）

程序性记忆，封装特定任务的工作流指令：

- 启动时扫描所有 Skill 描述，追加到系统提示词（Agent 始终知道有哪些技能）
- 任务匹配时 Agent 自主调用 use_skill(name) 按需加载全文（节省 token）
- CLI 支持 /skill-name 手动触发

Persistent Memory（harness/storage/）

SQLite 会话历史作为上下文记忆，压缩摘要作为长期记忆注入。（MCP 协议支持待扩展。）

5. 执行层 Execution Backends

工具的最终执行落点，MyHarnessPy 将这些后端**统一包装进一个 tool runtime**，上层调用方式完全一致。

后端	实现	安全措施
Terminal	tools/builtin/shell.py	asyncio 子进程，参数数组，防命令注入
FileSystem	tools/builtin/read_file.py / search.py	路径遍历防护，正则超时保护
External API	可通过 shell / 自定义工具扩展	由工具实现方负责认证
Sub-agents	待扩展	—

6. 状态层 State & Persistence

MyHarnessPy 不是无状态系统。会话、消息、检查点都进持久化存储。

SQLite 结构（`harness/storage/backends/sqlite.py`）

```
sessions(session_id, messages JSON, created_at, updated_at, metadata JSON)
checkpoints(checkpoint_id, session_id, round_index, state, messages JSON)
```

- 默认路径： `./harness.db`（可通过 `config.yaml` 的 `storage.path` 修改）
- `messages` 字段存完整 JSONL 序列，保留 `TextBlock` / `ThinkingBlock` / `ToolCallBlock` / `ToolResultBlock` 全部类型
- Checkpoint 支持轮级快照与回滚

MemoryStore（`harness/storage/backends/memory.py`）

测试与临时会话使用，接口与 SQLite 完全一致。

7. 模型层 Model / Provider Layer

Provider Runtime Resolver（`harness/llm/registry.py`）负责将配置中的 provider 名称解析为具体实现，统一认证、base URL 和 API mode：

```
build_provider(cfg)
  → "anthropic"           → AnthropicProvider   → anthropic_messages API
  → "openai"              → OpenAIProvider    → chat_completions API
  → "openai-compatible"   → OpenAIProvider    → 自定义 base_url（OpenRouter / 代理）
```

统一接口：

- `provider.chat(messages, tools)` — 主推理调用
- `provider.complete(prompt)` — 摘要生成（COMPRESS 专用，可绑定小模型）

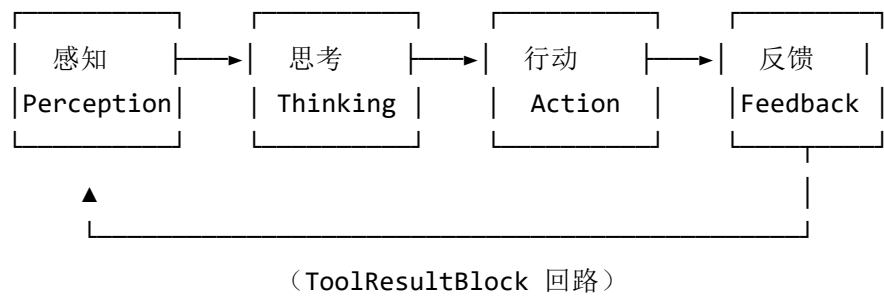
多 Provider 可在 `config.yaml` 中并行配置；Persona 可绑定特定 Provider；压缩摘要可独立指定 `summary_provider`。

8. 安全层 Security Boundary

安全不是单点，而是**多层边界**，贯穿整个调用栈：

边界	位置	实现
AUTH	会话与路由层	API Key 认证（通过 config.yaml / 环境变量注入）
APPROVAL	会话与路由层	POST /sessions/{id}/confirm / deny 危险操作审批接口
SHELL SAFETY	执行层	命令参数以数组传递，禁止字符串拼接，防命令注入
OUTPUT LIMITS	能力层	工具输出硬上限，超限走 OverflowStore 引用，防 token 爆炸
PERSONA GUARD	会话与路由层	allowed_tools 限定工具权限，非法工具调用被 registry 拒绝
MSG PROTOCOL	Agent Runtime	validate_message_sequence() 强制 ToolCall/ToolResult 配对

感知·思考·行动·反馈 映射



感知 (Perception)

来源	实现
用户输入	engine.send_message() → _messages 追加
会话历史	storage.load() 恢复持久化消息
系统提示词	load_persona() + build_skill_system_addendum()

来源	实现
上轮工具输出	ToolResultBlock — 行动→感知的核心回路
压缩摘要	compression.auto_compress() 后重注入

思考 (Thinking)

模块	作用
provider.chat(messages, tools)	核心推理入口，输出 ToolCallBlock / TextBlock
Extended Thinking (Anthropic)	链式思考暴露 ThinkingBlock ，可观测
Skill 描述 in 系统提示词	Agent 自主匹配任务，决定调用哪个 Skill
LoopDetector	重复推理模式检测，注入突破提示
Persona.allowed_tools	约束思考的工具边界

行动 (Action)

机制	作用
asyncio.gather()	同轮多工具真并发执行
Tool runtime	Terminal / FileSystem / SkillLoader 统一包装
OverflowStore	超限输出存引用，返回 ref ID

反馈 (Feedback)

机制	作用
ToolResultBlock → _messages	内部回路：行动输出立即成为下轮感知输入
validate_message_sequence()	保证 ToolCall/ToolResult 配对完整性
WebSocket 推送	外部回路：每条新消息实时推送用户
storage.save()	loop 结束后持久化，支持会话恢复
EventEmitter	结构化 JSON 日志，每个决策点可追溯

关键数据流

用户输入

- GW (api/rest.py 或 ws.py) 协议转换
- SessionRunner.send_message(text)
 - 若 RUNNING → 入队 _intervention_queue
 - 若 WAITING → 追加 Message → RUNNING → fire _run_loop_guarded()
- AIAgent 主循环 (loop.py)
 - PROMPT 组装: load_persona() + build_skill_system_addendum()
 - COMPRESS: maybe_compress()
 - 调模型: provider.chat(messages, tools) → assistant_msg
 - 执行工具: execute_all(tool_calls) → tool_result_msg
 - validate_message_sequence()
 - 原子追加: on_message(assistant) + on_message(tool_result)
 - 若有工具调用则继续下一轮; 否则 drain queue → return
- storage.save(session_id, messages)
- WebSocket 推送每条新消息

核心设计原则

原则	说明
消息协议优先	validate_message_sequence() 在每次 LLM 调用前强制校验
工具不缓存	每轮 registry.discover() 重新加载, 支持动态注册
锁外异步	asyncio.Lock 只保护变量读写, LLM 调用和工具执行在锁外
取消即时	cancel_event 与 LLM 调用并发等待, 毫秒级响应
压缩后重注入	Auto 压缩后强制重注系统提示词 + 任务目标, 防 AI 失忆
Persona 三合一	一个文件同时控制: 系统提示词 + 工具权限 + LLM Provider
Skill 按需加载	描述常驻提示词, 全文仅 use_skill 调用时加载, 节省 token
统一 tool runtime	执行后端差异对上层透明, 工具调用方式完全一致